TL-Engine – Full List of Methods (v1.09)

This is a list of methods available in the TL-Engine (as of version 1.09). You do not need to know or understand the whole list - use it for reference when writing your TL-Engine programs.

Models and Cameras

```
////////////////////////////////////
// Position

// Get current X position
float GetX();

// Get current Y position
float GetY();

// Get current Z position
float GetZ();

// Set the X position
void SetX( const float fX );

// Set the Y position
void SetY( const float fY );

// Set the Z position
void SetZ( const float fZ );

// Set the X, Y & Z positions
void SetPosition( const float fX, const float fY, const float fZ );


////////////////////////////////////
// Movement

// Move in the X direction by the given amount
void MoveX( const float fX );

// Move in the Y direction by the given amount
void MoveY( const float fY );

// Move in the Z direction by the given amount
void MoveZ( const float fZ );

// Move in the X, Y & Z directions by the given amounts
void Move( const float fX, const float fY, const float fZ );


////////////////////////////////////
// Local Movement

// Move in the local X direction by the given amount
void MoveLocalX( const float fX );

// Move in the local Y direction by the given amount
void MoveLocalY( const float fY );

// Move in the local Z direction by the given amount
void MoveLocalZ( const float fZ );
```

```
// Move in the local X, Y & Z directions by the given amounts
void MoveLocal( const float fX, const float fY, const float fZ );
////////////////////////////////////
// Orientation

// Reset the orientations (rotations) to the default
void ResetOrientation();


////////////////////////////////////
// Rotation

// Rotate around the X axis by the given amount (in degrees)
void RotateX( const float fAlpha );

// Rotate around the Y axis by the given amount (in degrees)
void RotateY( const float fBeta );

// Rotate around the Z axis by the given amount (in degrees)
void RotateZ( const float fGamma );
```

Meshes

```
////////////////////////////////////
// Model creation interface

// Uses this mesh to create a model in the scene at a given position.
// Returns a pointer to the new model or 0 for an error.
IModel* CreateModel( const float fX, const float fY, const float fZ );

// Removes one of this mesh's models from the scene
void RemoveModel( const IModel* pModel );
```

3D Engine

```
// Starts the 3D engine in a window of the given dimensions. This can only
// be done once after creating a 3D engine.
void StartWindowed( const int iWidth, const int iHeight );

// Delete the 3D engine. Must call this before finishing the program
// to correctly clean up the memory used by the 3D engine.
void Delete();


// Returns true if the 3D engine is initialised and running. Will
// return false if the system or user has requested the engine to stop.
// Use this function to control the main loop of your application.
bool IsRunning();

// Request that the 3D engine stops - will ensure IsRunning returns
// false. Use this function to terminate the main loop of your
// application from other parts of the code.
void Stop();


// Returns true if the engine window/fullscreen is active, i.e. it is the
// foreground window that the user is working with. If this function
// returns false, then any keyboard/mouse input should be ignored - it is
```

```
// intended for another window.
bool IsActive();
////////////////////////////////////
// Media folder management

// Add a folder to the list of folders searched for media
void AddMediaFolder( const string& sFolder );

// Remove a folder from the list of folders searched for media, returns
// true on success (folder was found and removed)
bool RemoveMediaFolder( const string& sFolder );

// Clears the list of folders searched for media
void ClearMediaFolders();


////////////////////////////////////
/ Mesh creation interface

// Loads a 3D mesh from a file, returns a pointer to the mesh loaded or
// 0 for an error. Only loads the mesh data, does not actually create
// a model in the scene. Use the mesh function CreateModel to create
// models to place in the scene. Note that different 3D engines support
// different file types.
IMesh* LoadMesh( const string& sMeshFileName );

// Removes a mesh from the system, also removes any models in the
// scene that use the mesh.
void RemoveMesh( const IMesh* pMesh );


////////////////////////////////////
// Camera creation interface

// Creates a camera of a given type (kStatic [default], kFPS, or
// kTargeted), positioned at a given location. Returns a pointer to
// the new camera.
ICamera* CreateCamera( const ECameraType eCameraType
                       const float fX, const float fY, const float fZ );

// Removes a camera from the scene.
void RemoveCamera( const ICamera* pCamera );


////////////////////////////////////
// Rendering interface

// Draw everything in the scene from the viewpoint of the given camera.
// If no camera is supplied, the most recently created camera is used.
void DrawScene( const ICamera* pCamera );


////////////////////////////////////
// UI interface

// Returns true when a given key or button is first pressed down. Use
/ for one-off actions or toggles. Example key codes: kKey_A or
/ kMouse_LButton, see input.h for a full list.
bool KeyHit( const EKeyCode eKeyCode );

// Returns true as long as a given key or button is held down. Use for
// continuous action or motion. Example key codes: kKey_A or
/ kMouse_LButton, see input.h for a full list.
bool KeyHeld( const EKeyCode eKeyCode );
```

```
// Returns the X position of the mouse (relative to the top-left of the
// 3D engine window/fullscreen).
int GetMouseX();

// Returns the Y position of the mouse (relative to the top-left of the
// 3D engine window/fullscreen).
int GetMouseY();

// Returns the wheel position of the mouse
float GetMouseWheel();


// Returns the X mouse movement since the last call to this function
int GetMouseMovementX();

// Returns the Y mouse movement since the last call to this function
int GetMouseMovementY();


// Start capturing the mouse. The mouse cursor will be removed and all
// mouse movement captured by the 3D engine window. Allows direct mouse
// control for games that do not need a mouse cursor.
void StartMouseCapture();

// Stops capturing the mouse. See 'StartMouseCapture'.
void StopMouseCapture();
```